Solar Array Reference

Component Design Document

1 Description

Solar array reference angle guidance. Computes the array rotation angle that turns the array surface normal toward the sun in the reference attitude, or holds a commanded angle, and publishes it for the array drive controller. The tracking mode and the angles are commanded through a connector on GNC state transitions. Wraps the SolarArrayReferenceAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 6
- **Number of Invokee Connectors** - 4
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 3
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 3
- **Number of Packets** - *None*

3.2 Diagram

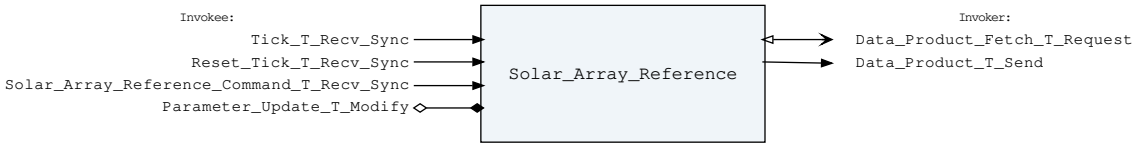


Figure 1: Solar Array Reference component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Solar Array Reference Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Reset_Tick_T_ Recv_Sync	recv_sync	Tick.T	-	1
Solar_Array_ Reference_ Command_T_Recv_ Sync	recv_sync	Solar_Array_ Reference_ Command.T	-	1
Parameter_ Update_T_Modify	modify	Parameter_ Update.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - Run the algorithm up to the current time.
- **Reset_Tick_T_Recv_Sync** - Zero the reference angle the algorithm retains from the previous tick, which is its fallback when the sun is aligned with the drive axis. The configuration is untouched.
- **Solar_Array_Reference_Command_T_Recv_Sync** - Command the tracking mode and the angles. The commanding component sends it on every GNC state transition into a state that controls the solar arrays, between ticks, and it reconfigures the algorithm for the ticks that follow. The component is not ticked, and not commanded, while the solar arrays are not controlled.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Solar Array Reference Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_ Fetch_T_Request	request	Data_Product_ Fetch.T	Data_Product_ Return.T	1

Data_Product_T_Send	send	Data_Product.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Solar Array Reference Set Id Bases Parameters

Name	Type
Data_Product_Id_Base	Data_Product_Types.Data_Product_Id_Base
Parameter_Id_Base	Parameter_Types.Parameter_Id_Base

Parameter Descriptions:

- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Parameter_Id_Base** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Solar Array Reference Map Data Dependencies Parameters

Name	Type
Navigation_Attitude_Id	Data_Product_Types.Data_Product_Id

Navigation_Attitude_Stale_Limit	Ada.Real_Time.Time_Span
Attitude_Reference_Id	Data_Product_Types.Data_Product_Id
Attitude_Reference_Stale_Limit	Ada.Real_Time.Time_Span
Sun_Direction_Body_Id	Data_Product_Types.Data_Product_Id
Sun_Direction_Body_Stale_Limit	Ada.Real_Time.Time_Span

Parameter Descriptions:

- **Navigation_Attitude_Id** - The external data product ID used to fetch the value for the 'Navigation_Attitude' data dependency.
- **Navigation_Attitude_Stale_Limit** - The stale limit used for the 'Navigation_Attitude' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **Attitude_Reference_Id** - The external data product ID used to fetch the value for the 'Attitude_Reference' data dependency.
- **Attitude_Reference_Stale_Limit** - The stale limit used for the 'Attitude_Reference' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **Sun_Direction_Body_Id** - The external data product ID used to fetch the value for the 'Sun_Direction_Body' data dependency.
- **Sun_Direction_Body_Stale_Limit** - The stale limit used for the 'Sun_Direction_Body' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the solar array reference algorithm with the default parameter values, sun tracking, and zero angles. The commanded mode and angles take effect when they are first received. The `init` subprogram takes no parameters.

3.6 Commands

The Solar Array Reference component has no commands.

3.7 Parameters

Parameters for the Solar Array Reference component.

Table 5: Solar Array Reference Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Drive_Axis	Packed_F32x3.T	[1.0,0.0,0.0]
0x0001 (1)	Surface_Normal	Packed_F32x3.T	[0.0,1.0,0.0]
0x0002 (2)	Alignment_Threshold	Packed_F32.T	(Value=>0.1)

Parameter Descriptions:

- **Drive_Axis** - [-] Solar array drive axis in body frame components. Must be finite, within 1e-3 of unit length, and orthogonal to the surface normal.
- **Surface_Normal** - [-] Solar array surface normal at zero rotation in body frame components. Must be finite, within 1e-3 of unit length, and orthogonal to the drive axis.

- **`Alignment_Threshold`** - [rad] Angle between the sun direction and the drive axis below which the array has no preferred angle and the reference holds the current angle. Must be in [1e-3, pi/2].

3.8 Events

The Solar Array Reference component has no events.

3.9 Data Products

Data products for the Solar Array Reference component.

Table 6: Solar Array Reference Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Reference_Angle	Packed_F32.T

Data Product Descriptions:

- **`Reference_Angle`** - [rad] Reference rotation angle of the solar array about its drive axis, wrapped to [-pi, pi].

3.10 Data Dependencies

Data dependencies for the Solar Array Reference component.

Table 7: Solar Array Reference Data Dependencies

Data Dependency Name	Type
Navigation_Attitude	Nav_Att_Output.T
Attitude_Reference	Att_Ref.T
Sun_Direction_Body	Packed_F32x3.T

Data Dependency Descriptions:

- **`Navigation_Attitude`** - This is the current navigation attitude estimate. Only the body attitude is used. It is fetched only in the sun tracking mode.
- **`Attitude_Reference`** - This is the reference attitude the body is slewing to. Only the reference attitude is used. It is fetched only in the sun tracking mode.
- **`Sun_Direction_Body`** - [-] This is the unit vector from the spacecraft to the sun in body frame components. It is fetched only in the sun tracking mode.

3.11 Packets

The Solar Array Reference component has no packets.

3.12 Faults

The Solar Array Reference component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Solar_ Array_ Reference_ Tests* Test Suite

This is a unit test suite for the Solar Array Reference component

Test Descriptions:

- **`Test`** - Run the algorithm on the sun tracking cases of the Python reference test to ensure the Ada to C to C++ integration is sound.
- **`Test_Tracking_Mode_Variants`** - Check the aligned sun fallback to the retained angle, the offset in sun tracking mode, and the commanded angle mode against the Python reference model.
- **`Test_Reset`** - Ensure the reset connector zeroes the retained reference angle and leaves the configuration in place.
- **`Test_Command_Retained`** - Ensure a command stays in effect on the ticks that follow it, including across a parameter update.
- **`Test_Invalid_Parameter`** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **`Test_Array_Angle_Range`** - Ensure the range of the commanded angle type is exactly the range the algorithm accepts, so command validation rejects what the algorithm would.
- **`Test_Specified_Angle_Without_Attitude`** - Ensure the specified angle mode runs without the attitude inputs and the sun tracking mode requires them.
- **`Test_Invalid_Data_Dependency`** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Att_Ref.T:

Table 8: Att_Ref Packed Record : 288 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sigma_Rn	Packed_F32x3.T	-	96	0	95
Omega_Rn_N	Packed_F32x3.T	-	96	96	191
Domega_Rn_N	Packed_F32x3.T	-	96	192	287

Field Descriptions:

- **Sigma_Rn** - *No description provided.*
- **Omega_Rn_N** - *No description provided.*
- **Domega_Rn_N** - *No description provided.*

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 9: Data_Product Packed Record : 1096 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_ Header.T	-	72	0	71	-
Buffer	Data_Product_ Types.Data_ Product_ Buffer_Type	-	1024	72	1095	Header. Buffer_Length

Field Descriptions:
- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 10: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:
- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 11: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:
- **Time** - The timestamp for the data product item.

- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 12: Data_Product_Return Packed Record : 1104 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_Status	Data_Product_Enums.Fetch_Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_Data_Product	Data_Product.T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Nav_Att_Output.T:

Table 13: Nav_Att_Output Packed Record : 352 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time_Tag	Long_Float	-inf to inf	64	0	63
Sigma_Bn	Packed_F32x3.T	-	96	64	159
Omega_Bn_B	Packed_F32x3.T	-	96	160	255
Veh_Sun_Pnt_Bdy	Packed_F32x3.T	-	96	256	351

Field Descriptions:

- **Time_Tag** - [s] Time tag of navigation solution
- **Sigma_Bn** - MRP attitude of body w.r.t. inertial frame
- **Omega_Bn_B** - [rad/s] Angular velocity of body w.r.t. inertial, body frame components
- **Veh_Sun_Pnt_Bdy** - Sun direction unit vector in body frame

Packed_F32.T:

Single component record for holding packed 32-bit floating point number.

Table 14: Packed_F32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Short_Float	-3.40282e+38 to 3.40282e+38	32	0	31

Field Descriptions:

- **`Value`** - The 32-bit floating point number.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 15: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
`Short_Float`	`-3.40282e+38 to 3.40282e+38`	32	3	96

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 16: Parameter Packed Record : 792 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_ Header.T	-	24	0	23	-
Buffer	Parameter_ Types. Parameter_ Buffer_Type	-	768	24	791	Header.Buffer_ Length

Field Descriptions:

- **`Header`** - The parameter header
- **`Buffer`** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 17: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:

- **`Id`** - The parameter identifier
- **`Buffer_Length`** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 18: Parameter_Update Packed Record : 824 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_Types.Parameter_Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_Enums.Parameter_Operation_Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate	8	16	23	-
Status	Parameter_Enums.Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter.T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Solar_Array_Reference_Command.T:

This is the command to the solar array reference guidance of one SADA. The commanding component sends it on every GNC state transition into a state that controls the solar arrays, so it always describes what the SADA does while it is controlled. In Auto_Track the array tracks the sun and the offset angle is added to the tracking angle. In Specified_Angle the array is held at the specified angle.

Table 19: Solar_Array_Reference_Command Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Mode	Solar_Array_Reference_Enums.Tracking_Mode.E	0 => Auto_Track 1 => Specified_Angle	8	0	7
Specified_Array_Angle	Solar_Array_Reference_Types.Array_Angle	-3.14159 to 3.14159	32	8	39
Offset_Angle	Solar_Array_Reference_Types.Array_Angle	-3.14159 to 3.14159	32	40	71

Field Descriptions:

- **Mode** - This mode selects what the SADA does. In Auto_Track it tracks the sun. In Specified_Angle it is held at the specified angle. The enumeration has exactly the modes the algorithm accepts.
- **Specified_Array_Angle** - [rad] The angle the array is held at in Specified_Angle. It is ignored in the other modes. The type carries the range the algorithm accepts.
- **Offset_Angle** - [rad] The offset added to the sun tracking angle in Auto_Track. It is ignored in the other modes. The type carries the range the algorithm accepts.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 20: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 21: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 22: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 23: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 24: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.

Solar_Array_Reference_Enums.Tracking_Mode.E:

Solar array reference tracking mode. Values match the C TrackingMode enumeration.

Table 25: Tracking_Mode Literals:

Name	Value	Description
Auto_Track	0	The reference angle turns the array surface normal toward the sun.
Specified_Angle	1	The reference angle is the specified array angle.